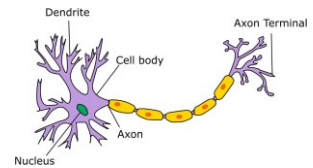
Neural Network

Table of contents

:: Neural Network

Structure of Neurons in human brain

The human brain can be described as a biological neural network—an interconnected web of neurons transmitting elaborate patterns of electrical signals. Dendrites receive input signals and, based on those inputs, fire an output signal via an axon[1].



- ***Dendrite***: *It receives signals from other neurons*
- ***Cell body***: *It sums all the incoming signals to generate input*
- ***Axon***: *When the sum reaches a threshold value, neuron fires and the signal travels down the axon to the other neurons*

http://neuralnetworksanddeeplearning.com/chap1.html

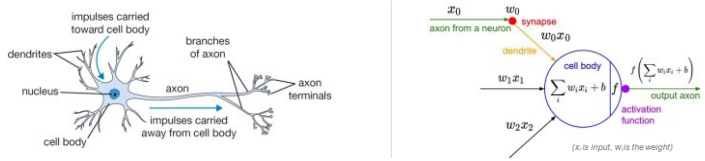
:: Neural Network

The neuron is the basic computational unit of the brain

Neuron , often called a node or unit, receives input from some other neurons, or from an external source and computes an output. Each input has an associated weight (w), which is assigned on the basis of its relative importance to other inputs. The node applies a function to the weighted sum of its inputs.

The idea is that the synaptic strengths (the weights *w*) are learnable and control the strength of influence and its direction: excitory (positive weight) or inhibitory (negative weight) of one neuron on another. In the basic model, the dendrites carry the signal to the cell body where they all get summed. If the final sum is above a certain threshold, the neuron can *fire*, sending a spike along its axon[1].

Artificial neural networks are computing systems inspired by the biological neural networks.

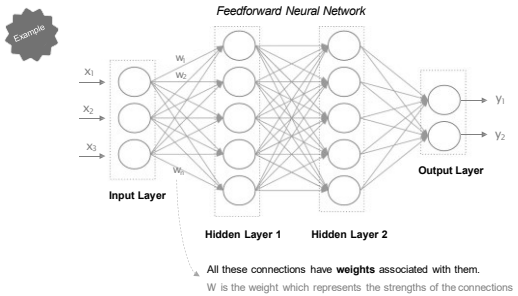


biological neuron (left) and a common mathematical model (right)

:: Neural Network

Artificial neural network (ANN)

Inspired by the human brain, an ANN is comprised of a network of artificial neurons (also known as "nodes"). These nodes are connected to each other, and the strength of their connections to one another is assigned a value based on their strength. If the value of the connection is high, then it indicates that there is a strong connection[1]. The network is trained by iteratively modifying the strengths of the connections so that given inputs map to the correct response[2].



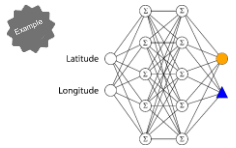
A feedforward neural network is an artificial neural network where connections between the units do not form a cycle. In this network, the information moves in only one direction, forward, from the input nodes, through the hidden nodes (if any) and to the output nodes. There are no cycles or loops in the network.

All these connections have **weights** associated with them.
W is the weight which represents the strengths of the connections

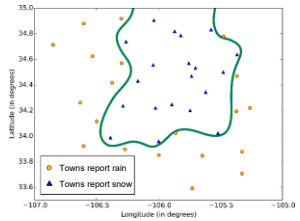
:: Neural Network

Neural Network is best used...[1]

- For modeling highly nonlinear systems
- When data is available incrementally and you wish to constantly update the model
- When there could be unexpected changes in your input data
- When model interpretability is not a key concern



Neural network for classification problem



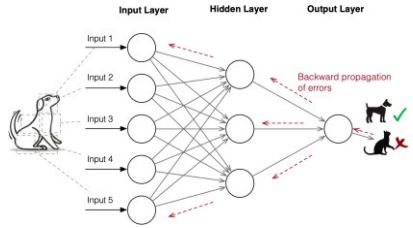
The boundaries learned by neural networks can be complex and irregular

This high performance doesn't come for free, though. Neural networks can take a long time to train, particularly for large data sets with lots of features. They also have more parameters than most algorithms, which means that parameter sweeping expands the training time a great deal [2]

:: Neural Network

Most modern deep learning models are based on an Artificial Neural Network [1]

Neural networks and deep learning currently provide the best solutions to many problems in image recognition, speech recognition, and natural language processing, where they have produced results comparable to and in some cases superior to human experts.

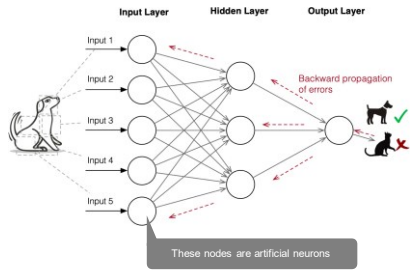


This simplified conceptual diagram illustrates how a simple neural network works

:: Neural Network

Artificial neurons are most basic units of information processing in an artificial neural network.

Each neuron takes information from a set of neurons, processes it, and gives the result to another neuron for further processing.

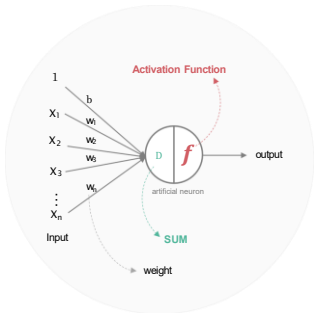


:: Neural Network

How does artificial neuron work ?

Let's take a closer look at artificial neuron.

- The artificial neuron receives one or more inputs and sums them to produce an output . Each of neurons has some weight associated with it.

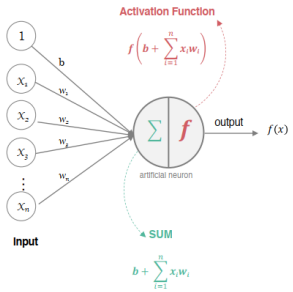


- Usually each input is separately weighted, and the sum is passed through a non-linear function known as an activation function or transfer function. [1]

Basic structure of an artificial neuron

:: Neural Network

How does artificial neuron work ? - continued



1. **All the inputs x are multiplied with their weights w, and added together**

$$weighted\ sum = b*1 + x_1*w_1 + \ldots + x_n*w_n = b + \sum_{i=1}^{n} x_i w_i$$

2. **And then the weighted sum is passed to activation function**

The activation function is set of the transfer function used to get desired output. There are many types of activation function. One of the simplest would be step function.

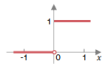
$$f(x) = f\left(b + \sum_{i=1}^{n} x_i w_i\right)$$

$$Step\left(b + \sum_{i=1}^{n} x_i w_i\right)$$

e.g. use **Heaviside Step function** as Activation Function

A step function will typically output a 1 if the input is higher than a certain threshold, otherwise it's output will be 0.

$$Hevside(x) = \begin{cases} 1, & for\ x \geq 0 \\ 0, & for\ x < 0 \end{cases}$$

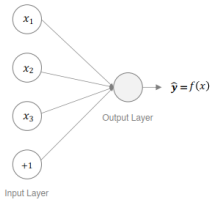


:: Neural Network

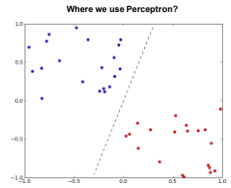
Single-layer Perceptron

Let's take a look at Perceptron. As a linear classifier, the Perceptron is the simplest feedforward neural network. It's also termed the single-layer perceptron, to distinguish it from a multilayer neural network. It does not contain any hidden layers, which means it only consists of a single layer of output nodes.

Perceptron can be defined as a single artificial neuron that computes its weighted input, and uses a threshold activation function to output a quantity. It's also called as TLU (Threshold Logic Unit).



Use Perceptron to classify data



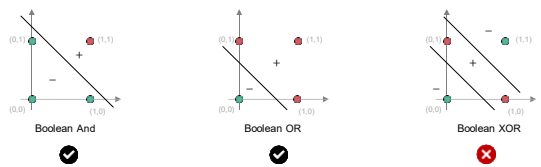
Perceptron is usually used to classify the data into two parts. Therefore, it is also known as a Linear Binary Classifier[1].

:: Neural Network

Limitation of Perceptron

A single layer perceptron can only learn linearly separable problems. If the vectors are not linearly separable, learning will never reach a point where all vectors are classified properly.

linearly separable ?



A single perceptron can only learn linearly separable functions.

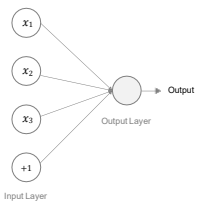
Boolean And function, Boolean OR function are linearly separable, whereas XOR (exclusive or) function is not linearly separable (Its positive and negative instances can't be separated by a line or hyperplane).

Even when the data set is linearly separable, there may be many solutions, and which one is found will depend on the initialization of the parameters and on the order of presentation of the data points. Furthermore, for data sets that are not linearly separable, the perceptron learning algorithm will never converge [1].

:: Neural Network

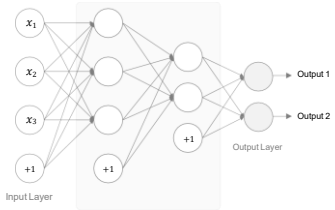
Multi-layer Perceptron can solve non-linear separable problem

A multi-layer perceptron (MLP) is a class of feedforward artificial neural network. Given a set of features $X = x_1, x_2, \ldots x_m$ and a target y, Multi-layer Perceptron can learn a non-linear function approximator for either classification or regression[1].



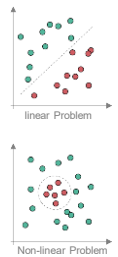
Single-Layer Perceptron

It does not contain any hidden layer. It performs binary classification (either this or that). But it can't learn non-linearly separable problem.



Multi-Layer Perceptron (MLP)

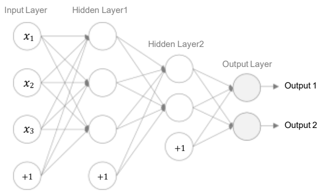
It contains hidden layers. It's able to learn not only linear problems but also non-linear problems. In most cases the data is not linearly separable. An MLP neuron is free to either perform classification or regression, depending upon its activation function[2]



[1] Source: http://scikit-learn.org/stable/modules/neural_networks_supervised.html

:: Neural Network

Let's dive into Multi-layer Perceptron



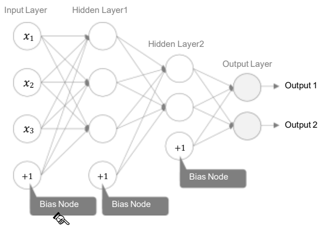
- **Input layer**
  The Input nodes provide information from the outside world to the network and are together referred to as the "Input Layer". No computation is performed in any of the Input nodes – they just pass on the information to the hidden nodes[1].
- **Hidden Layers**
  The hidden layer is the collection of neurons which has activation function applied on it and it is an intermediate layer found between the input layer and the output layer. Its job is to process the inputs obtained by its previous layer[2]. They perform computations and transfer information from the input nodes to the output nodes[3]. Also there can be multiple hidden layers in a Neural Network.
- **Output Layer**
  The Output nodes are collectively referred to as the "Output Layer" and are responsible for computations and transferring information from the network to the outside world[4].

Source: https://medium.com/@ariesiitr/an-artificial-neural-network-ann-is-a-computational-model-that-is-inspired-by-the-way-biological-c17b07166d4c

:: Neural Network

Let's dive into Multi-layer Perceptron

Bias Node / Bias Unit are included in the input layer and hidden layers. The Bias node has a value of 1.



Biases are added to the sums calculated at each node (except input nodes) during the feedforward phase.

$b + \sum_{012}^{3} x_i w_i$

Bias nodes are added to increase the flexibility of the model to fit the data

The introduction of Bias neurons allows you to shift the transfer function curve horizontally (left/right) along the input axis while leaving the shape/curvature unaltered. This will allow the network to produce arbitrary outputs different from the defaults and hence you can customize/shift the input-to-output mapping to suit your particular needs[1]. The negative of a bias is sometimes called a threshold (Bishop, 1995a).

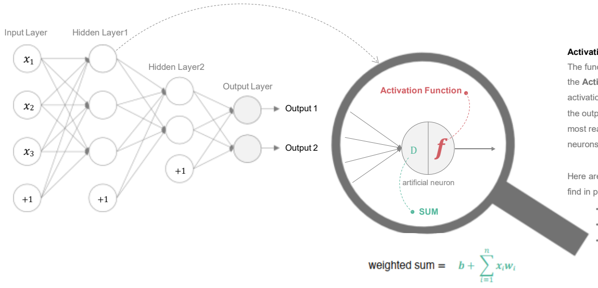
A simpler way to understand what the bias is: it is somehow similar to the constant b of a linear function $y = ax + b$. It allows you to move the line up and down to fit the prediction with the data better. Without b the line always goes through the origin (0, 0) and you may get a poorer fit[2].

For more detailed explanation on why Bias is used, Click this link

:: Neural Network

Let's dive into Multi-layer Perceptron

The activation function of a node defines the output of that node. Every activation function takes a single number and performs a certain fixed mathematical operation on it.



Activation Function

The function ***f*** is non-linear and is called the **Activation Function**. The purpose of the activation function is to introduce non-linearity into the output of a neuron. This is important because most real world data is non linear and we want neurons to *learn* these non linear representations.

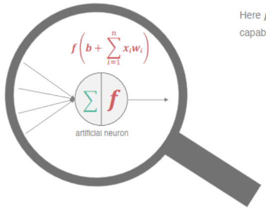
Here are some activations functions you will often find in practice:

- Sigmoid
- Tanh
- ReLU

:: Neural Network

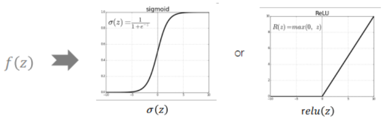
Why is Activation Functions used in neural networks?

- Most important reason is to **introduce nonlinearity** to the network *.
- A neural network without a non-linear activation function is essentially just a linear regression model which is less useful
- The activation function does the non-linear transformation to the input making it capable to learn and perform more complex tasks[1].



Here f is known an **activation function**. This makes a Neural Network extremely flexible and imparts the capability to estimate complex non-linear relationships in data [3].

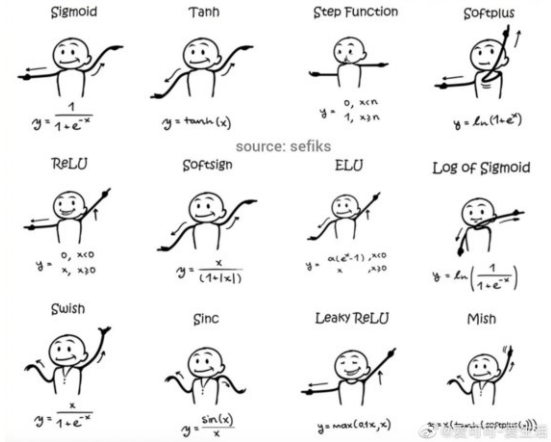
For example, we can use Sigmoid or ReLu as activation function



Note: The activation function can be a linear function (which represents straight line or planes) or a non-linear function (which represents curves). Most of the time, the activation functions used in neural networks will be non-linear.

Source: www.analyticsvidhya.com/blog/2016/03/introduction-deep-learnfundamentals-neural-networks

:: Neural Network



:: Neural Network

Some activation functions

Name	Plot	Equation	Derivative (with respect to x)	Range
Identity function		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$	$\{0, 1\}$
Logistic		$f(x) = \frac{1}{1+e^{-x}}$	$f'(x) = f(x)(1-f(x))$	$(0, 1)$
TanH		$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$	$(-1, 1)$
Rectified linear unit (ReLu)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$[0, \infty)$
Leaky ReLU		$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Parameteric rectified linear unit (PReLU)		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Randomized leaky rectified linear unit (RReLU)		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$ [1]	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Exponential linear unit (ELU)		$f(\alpha, x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} f(\alpha, x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\alpha, \infty)$

- **Identify function / linear activation function**
  The function is a line or linear. The function is then used to implement a standard linear regression model for basic forecasting problems and therefore rarely used in neural networks[1]

- **Step Function**
  A step function originally used in a perceptron. The binary step function is extremely simple. It can be used while creating a binary classifier, which outputs 1 if the input is above threshold and otherwise outputs 0 .

  The gradient of the step function is zero. So it's not useful for back-propagation algorithm of neural network

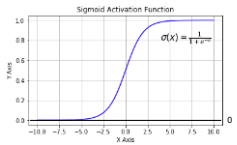
Next let's look into the Most popular types of Activation functionsin more details

- Sigmoid (Logistic)
- TanH
- ReLU
- SoftMax

http://www.srutsj.in/blog/research/designing-neural-network-what-activation-function-you-plan-to-implement/

:: Neural Network

Most popular types of Activation functions- Sigmoid



The Sigmoid Function curve looks like a S-shape. It is also known as Logistic Activation Function.

Mathematically it is represented as

$$f(x) = \frac{1}{1+e^{-x}}$$

This is a smooth function and is continuously differentiable which is is necessary for enabling gradient-based optimization methods.

Sigmoid takes a real-valued number and "squashes" it into range between **0 and 1**. In particular

- large negative numbers become 0
- large positive numbers become 1

It is also used in the output layer where our end goal is to predict probability. Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice.

In practice, the sigmoid non-linearity has recently fallen out of favor and it is rarely ever used. It has two major drawbacks [1] :

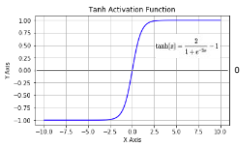
- **Vanishing gradients** (*Sigmoids saturate and kill gradients*)
  The neuron's activation saturates at either tail of 0 or 1. The curve is flat near 0 and 1. The gradient at these regions is almost zero. if the local gradient is very small, it will effectively "kill" the gradient during backpropagation[2]. Thus, the weights in these neurons do not update. Not only that, the weights of neurons connected to such neurons are also slowly updated. This problem is also known as vanishing gradient [3].

- **Sigmoid outputs are not zero-centered**
  This could introduce undesirable zig-zagging dynamics in the gradient updates for the weights [4]

1,2,4 source: http://cs231n.github.io/neural-networks-1/
3 Source: https://www.learnopencv.com/understanding-activation-functions-in-deep-learning/

:: Neural Network

Most popular types of Activation functions- Tanh



Tanh function is also like sigmoid but better.

- The range of tanh is from (-1 to 1)
- The range of sigmoid is from (0 to 1)

Mathematically it is represented as

$$Tanh\,(x) = 2 \cdot \sigma\,(2x) - 1$$

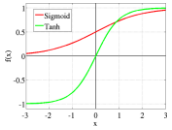
- Where σ(x) is the sigmoid function

You can think of a tanh function as two sigmoids put together[2].

Tanh function takes a real-valued input and "squashes" it into range between **-1 and 1**.

Unlike sigmoid, tanh outputs are zero-centered since the scope is between -1 and 1. The negative inputs considered as strongly negative, zero input values mapped near zero, and the positive inputs regarded as positive[1].

In practice, tanh is preferable over sigmoid



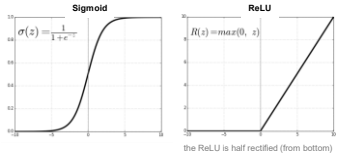
The tanh function also suffers from the vanishing gradient problem and therefore kills gradients when saturated [3].

1,2,3 Source: https://www.learnopency.com/understanding-activation-functions-in-deep-learning/

:: Neural Network

Most popular types of Activation functions- ReLU

The ReLU (Rectified Linear Unit) is the **most used activation** function in the world right now. Since, it is used in almost all the convolutional neural networks or deep learning. It takes a real-valued input and thresholds it at zero (replaces negative values with zero)



the ReLU is half rectified (from bottom)

$f(x \doteq \max(0, x)$

This means that when the input x < 0 the output is 0 and if x > 0 the output is x. This activation makes the network converge much faster. It does not saturate which means it is resistant to the vanishing gradient problem at least in the positive region (when x > 0), so the neurons do not backpropagate all zeros at least in half of their regions.

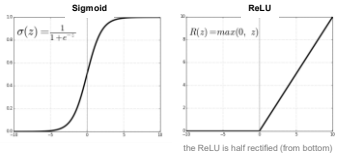
ReLU is computationally very efficient because it is implemented using simple thresholding.

1 2, 4 source: http://cs231n.github.io/neural-networks-1/
3 source: https://www.learnopencv.com/understanding-activation-functions-in-deep-learning/

:: Neural Network

Most popular types of Activation functions- ReLU(continued)

But there are few drawbacks of ReLU neuron :



the ReLU is half rectified (from bottom)

$f(x \doteq \max(0, x)$

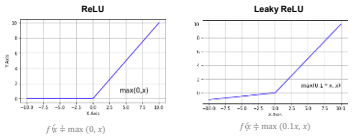
- **Not zero-centered**
  The outputs are not zero centered similar to the sigmoid activation function
- **Unfortunately, ReLU units can be fragile during training and can "die"**
  The issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly. if x < 0 during the forward pass, the neuron remains inactive and it kills the gradient during the backward pass. Thus weights do not get updated, and the network does not learn. For example, you may find that as much as 40% of your network can be "dead" (i.e. neurons that never activate across the entire training dataset) if the learning rate is set too high

1 2, 4 source: http://cs231n.github.io/neural-networks-1/
3 source: https://www.learnopencv.com/understanding-activation-functions-in-deep-learning/

:: Neural Network

Most popular types of Activation functions- Leaky ReLU

To address the vanishing gradient issue in ReLU activation function when x < 0, we have something called **Leaky ReLU** which was an attempt to fix the dead ReLU problem.



$f(x \doteq \max(0, x)$ $f(x \doteq \max(0.1x, x)$

$f(x \doteq \max(0.1x, x)$

Leaky ReLU is one attempt to fix the "dying ReLU" problem. Instead of the function being zero when x < 0, a leaky ReLU will instead have a small negative slope (of 0.01, or so)[1].

Some people report success with this form of activation function, but the results are not always consistent[2].

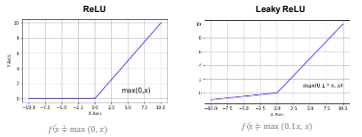
The idea of leaky ReLU can be extended even further. Instead of multiplying x with a constant term we can multiply it with a hyperparameter which seems to work better the leaky ReLU.

1 2, source: http://cs231n.github.io/neural-networks-1/
3,4 Source: https://www.learnopencv.com/understanding-activation-functions-in-deep-learning/

:: Neural Network

Most popular types of Activation functions- PReLU

The idea of leaky ReLU can be extended even further. Instead of multiplying x with a constant term we can multiply it with a hyperparameter which seems to work better the leaky ReLU. This extension to leaky ReLU is known as **PReLu** (Parametric ReLU).

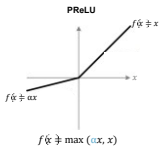


$f(x \doteq \max(0, x)$ $f(x \doteq \max(0.1x, x)$

$f(x \doteq \max(\alpha x, x)$

Where α is a hyperparameter which can be learned since you can backpropagate into it.

This gives the neurons the ability to choose what slope is best in the negative region, and with this ability, they can become a ReLU or a leaky ReLU[4].



$f(x \doteq \max(\alpha x, x)$

In summary, it is better to use ReLU, but you can experiment with Leaky ReLU or Parametric ReLU to see if they give better results for your problem

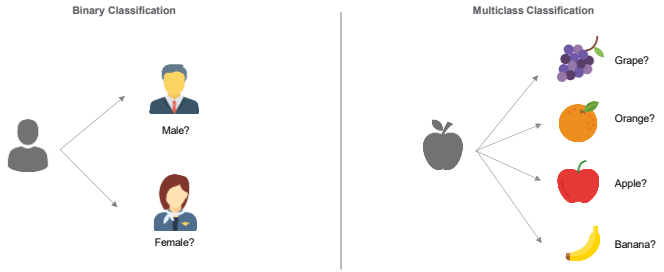
1 2, source: http://cs231n.github.io/neural-networks-1/
3,4 Source: https://www.learnopencv.com/understanding-activation-functions-in-deep-learning/

:: Neural Network

:: Neural Network

Most popular types of Activation functions– SoftMax

The softmax function is used in various **multiclass classification** methods, such as softmax regression, naive Bayes classifiers, and artificial neural networks.

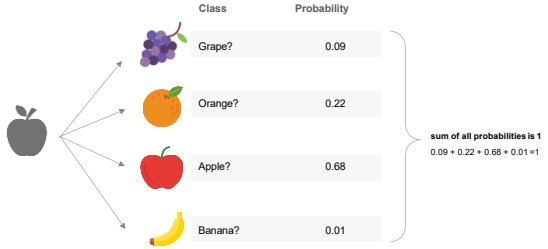


:: Neural Network

Most popular types of Activation functions– SoftMax (continued)

Softmax assigns decimal probabilities to each class in a multi-class problem. Those decimal probabilities must add up to 1.0. [1] The softmax function is often used in the final layer of a neural network-based classifier [2]. The Softmax layer must have the same number of nodes as the output layer [3].

Softmax might produce the following likelihoods of an image belonging to a particular class:



:: Neural Network

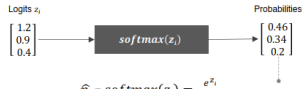
Most popular types of Activation functions– SoftMax (continued)

The Softmax function is a generalization of Logistic Regression. It converts linear inputs to probabilities. Softmax is implemented through a neural network layer just before the output layer [1].

- **Logistic Function**
  It's used for binary classification

$$f(x) = \frac{1}{1 + e^{-x}}$$

- **SoftMax Function**
  It's used in multiclass classification



$$\hat{p}_i = softmax(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

for $i = 1, \ldots, j$

The softmax function squashes the outputs of each unit to be between 0 and 1, just like a Logistic/Sigmoid function.

Unlike logistic regression which is for the binary classification, softmax allows for classification into any number of possible classes. Softmax classifier provides "probabilities" for each class. The total sum of probabilities for each class is equal to 1. In this example, 0.46+0.34+0.2 =1

:: Neural Network

Most popular types of Activation functions– SoftMax (continued)

Let's see an example on training a network to recognize and classify handwritten digits from images. The network would have ten output units, one for each digit 0 to 9. Then if you fed it an image of a number 9 (see below), the output unit corresponding to the digit 9 would be activated.

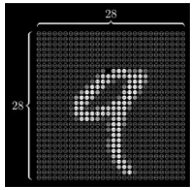
Example

Handwritten digits

Famous MNIST dataset for handwritten digits recognition

Each image is 28 pixels by 28 pixels which has been flattened into 1-D array of size 784.

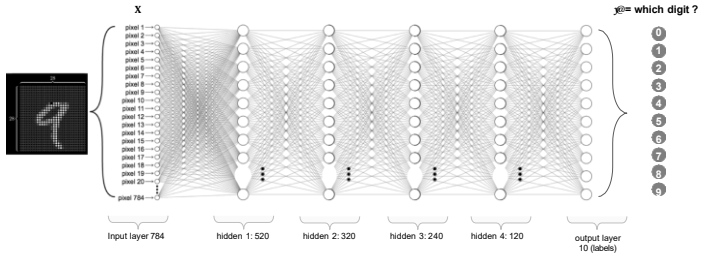
We generally think of MNIST data as being a collection of 784-dinenbsional vectors



28x28 pixels = 784

:: Neural Network

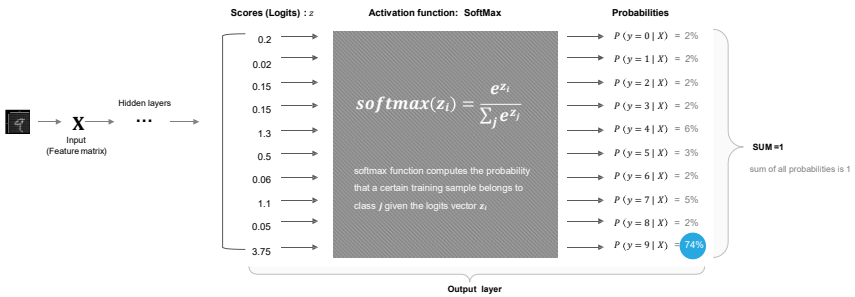
Most popular types of Activation functions– SoftMax (continued)



:: Neural Network

Most popular types of Activation functions– SoftMax (continued)

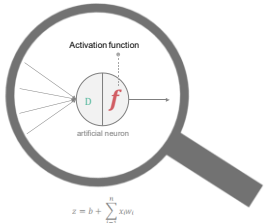
In order to convert the score matrix Z to probabilities, we use Softmax function.



z_i is a vector of the inputs to the output layer (if you have 10 output units, then there are 10 elements in z) [1]. j indexes the output units, so j = 1, 2, ..., i. Softmax function provides us with a vector of probabilities of each class label for a given observation. The image looks the most like the digit 9 because the probability of being 9 is highest

:: Neural Network

Which Activation Functions $f(z)$ Should I use ?



$$z = b + \sum_{i=1}^{n} x_i w_i$$

$$f(z) = f(b + \sum_{i=1}^{n} x_i w_i)$$

In Hidden Layer

- By Default use **ReLu** function
- In case that ReLu doesn't provide the optimum results, then try other activations like Leaky Relu or Maxout function
- Always keep in mind that ReLU function should only be used in the hidden layers
- Never use **Sigmoid and TanH** fucntion due to the vanishing gradient problem

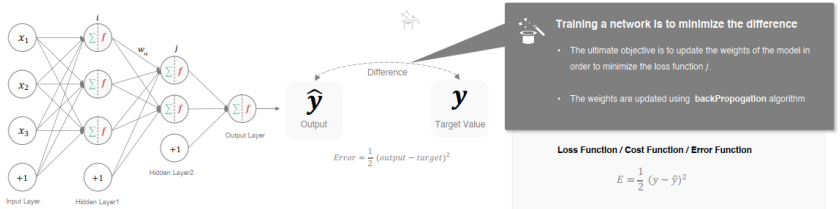
In Output Layer

- Use **SoftMax function** for classification problem
- Use **Linear function** for regression problem

:: Neural Network

How does the network learn ?

We'll use sum of square errors, the difference between target (known) data and the value estimated by our network, to compute an overall cost and we'll try to minimize it [1]. In a neural network, we're trying to minimize the error with respect to the weights. We want to find the optimal combination of weights and bias that minimize the loss function over the training set.



Training a network is to minimize the difference

- The ultimate objective is to update the weights of the model in order to minimize the loss function /.
- The weights are updated using **backPropogation** algorithm

Loss Function / Cost Function / Error Function

$$E = \frac{1}{2}(y - \hat{y})^2$$

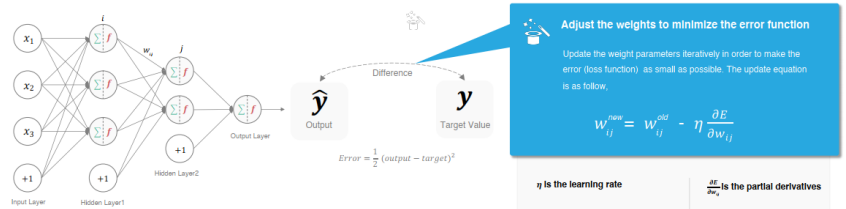
- Loss Function is used to measure exactly how wrong our predictions are.
- The loss is high when the neural network makes a lot of mistakes, and it is low when it makes fewer mistakes[2].

[1] ... src: http://www.bogotobogo.com/python/scikit-learn/scikit-learn-Artificial-Neural-Network-ANN-4-Backpropagation.php

:: Neural Network

How does the network learn ?

We're going to make the loss function/error function as small as possible by **finding the optimal weights value**. The weights and bias inside the network get updated after many iterations.



Adjust the weights to minimize the error function

Update the weight parameters iteratively in order to make the error (loss function) as small as possible. The update equation is as follow,

$$w_{ij}^{new} = w_{ij}^{old} - \eta \frac{\partial E}{\partial w_{ij}}$$

η is the learning rate

The choice of learning rate is important, since a high value can cause too strong a change, causing the minimum to be missed, while a too low learning rate slows the training unnecessarily[1].

$\frac{\partial E}{\partial w_{ij}}$ is the partial derivatives

$\frac{\partial E}{\partial w_{ij}}$ is the partial derivatives of the loss function with respect to the weight w_{ij}

[1] Source : https://www.bogotobogo.com/python/scikit-learn/scikit-learn-Artificial-Neural-Network-ANN-4-Backpropagation.php

:: Neural Network

How do you pick these optimal weights which can minimize the loss function ?

- **Use gradient descent algorithm to find the weights which can bring the error down**

  In the figure below, the loss function is shaped like a bowl. At any point in the training process, the partial derivatives of the loss function with respect to the weights is nothing but the slope of the bowl at that location. One can see that by moving in the direction predicted by the partial derivatives, we can reach the bottom of the bowl and therefore minimize the loss function. This idea of using the partial derivatives of a function to iteratively find its local minimum is called the **gradient descent**[1]

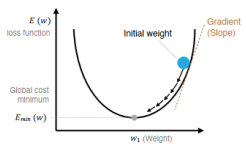
- **The new weights is obtained by moving the direction of the negative gradient along the multidimensional error surface**

Update the weights

$$w_{ij} = w_{ij} - \eta * \frac{\partial E}{\partial w_{ij}}$$

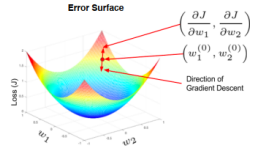
$$\text{Gradient} = \frac{\text{change in error}}{\text{change in weight}} = \frac{\Delta error}{\Delta weight} = \frac{\partial E}{\partial w}$$

Error Surface



Error surface of a linear neuron with one weight

Error Surface



Error surface of a linear neuron with two input weights

☛ An animation example for gradient descent

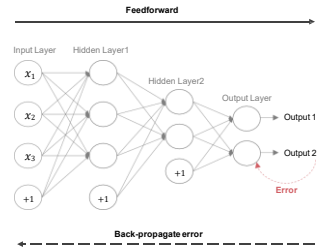
[1] Source : https://www.learnopencv.com/understanding-activation-functions-in-deep-learning/

:: Neural Network

Backpropagation algorithm

- **In Artificial neural networks the weights are updated using a method called Backpropagation.**

  The partial derivatives of the loss function with respect to the weights are used to update the weights. In a sense, the error is backpropagated in the network using derivatives. This is done in an iterative manner and after many iterations, the loss reaches a minimum value, and the derivative of the loss becomes zero.[1]



To tune the weights between the hidden layer and the input layer, we need to know the error at the hidden layer, but we know the error only at the output layer. So we can take the errors at the output layer and proportionally propagate them backwards to the hidden layer[2].

The backpropagation Algorithms:

1. **Feed-forward computation**
2. **Back-propagate Error Signals**
3. **Calculate Parameter Gradients**
4. **Update Parameters**

   The algorithm is stopped when the value of the error function has become sufficiently small.

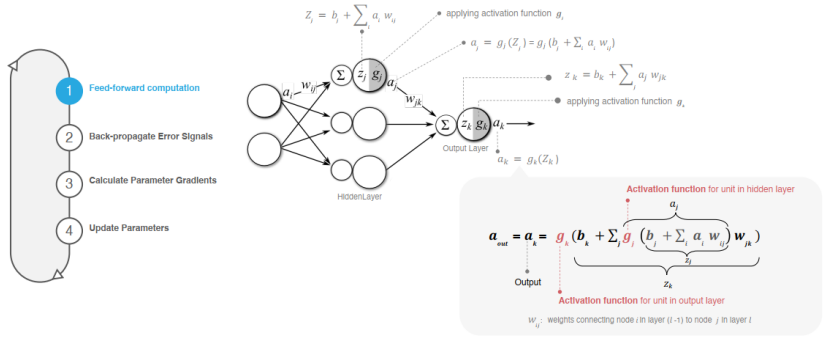
[1] Source : https://www.learnopencv.com/understanding-activation-functions-in-deep-learning/

:: Neural Network

Backpropagation algorithm

- **Propagation forward through the network to generate the output value(s)**

  When an input vector is presented to the network, it is propagated forward through the network, layer by layer, until it reaches the output layer. [1].

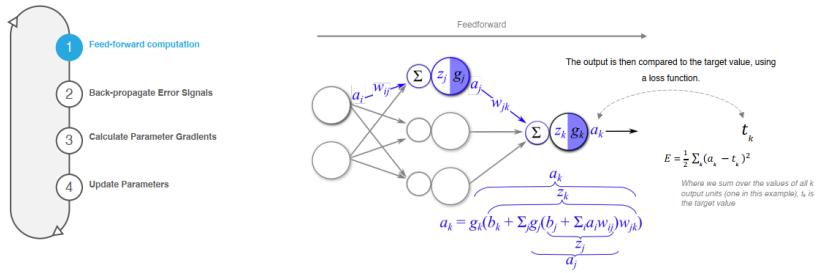


:: Neural Network

Backpropagation algorithm

- **Propagation forward through the network to generate the output value(s)**

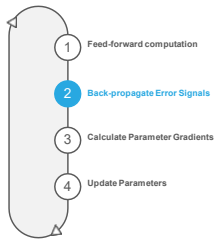
  When an input vector is presented to the network, it is propagated forward through the network, layer by layer, until it reaches the output layer. [1].

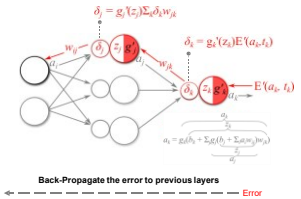


:: Neural Network

Backpropagation algorithm

- **Calculate output error *E* based on the predictions a and the target t**
- **Backward propagation of errors**
  - Backpropagation to the output layer
  - Backpropagation to the hidden layer





These error values will be used to calculate the gradient of the loss function.

- **Calculate the "Error term" δ_k for output node**

  Here the δ_k terms can be interpreted as the network output error after being back-propagated through the output activation function, thus creating an error "signal". [1]

$$\delta_k = g_k'(z_k)E'(a_k, tk) = g_k'(z_k)(a_k - t_k)$$

- **Calculate the "Error term" δ_j for the hidden layer node**

  Project δ_k back through w_{jk}, then through the activation function for the hidden layer via g_j' to give the error signal δ_j. [2]

$$\delta_j = g_j'(z_j)\sum_k \delta_k w_{jk}$$

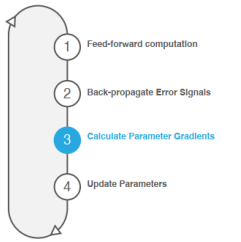
Note: The derivative of a function f e.g. will be denoted f' so, g_k' is the derivative of output layer activation function

:: Neural Network

Backpropagation algorithm

- **Calculate the gradients $\frac{\partial E}{\partial R}$ for the parameters (i.e. Compute the desired partial derivatives)**

  Training a neural network with gradient descent requires the calculation of the gradient of the error function E with respect to the weights and biases (collectively denoted θ) . Step 3 of the backpropagation algorithm is to calculate the gradients of the error function with respect to the model parameters at each layer l using the forward signals $\boldsymbol{a}_{l-1}$, and the backward error signals $\boldsymbol{\delta}_l$.[1]



Calculate the partial derivative of the error function E with respect to parameters

- **Gradients for Output Layer connection Weights**

$$\frac{\partial E}{\partial w_{jk}} = a_j \delta_k \quad \text{where} \quad \delta_k = g_k'(z_k)E'(a_k, tk) = g_k'(z_k)(a_k - tk)$$

- **Gradients for Hidden Layer Weights**

$$\frac{\partial E}{\partial w_{ij}} = a_i \delta_j \quad \text{where} \quad \delta_j = g_j'(z_j)\sum_{k,k} \delta_k \; w_{jk}$$

- **Gradients for Biases** (take the bias b_k as example)

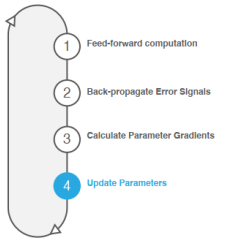
$$\frac{\partial E}{\partial b_k} = \delta_k$$

:: Neural Network

Backpropagation algorithm

- **Update parameters (Weights and Bias)**

Update the model parameters based on the gradients calculated in Step 3. Note that the gradients point in the direction in parameter space that will increase the value of the error function. Thus when updating the model parameters we should choose to go in the opposite direction. How far do we travel in that direction? That is generally determined by a user-defined step size (aka learning rate) parameter, η Thus given the parameter gradients and the step size, the weights and biases for a given layer are updated accordingly.[1]



Update the parameters using the calculated gradients

- **Update the parameter w_{jk}**

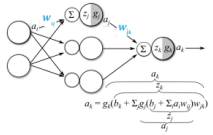
$$w_{jk} = w_{jk} - \eta \frac{\partial E}{\partial w_{jk}} \qquad \frac{\partial E}{\partial w_{jk}} = a_j \delta_k$$

- **Update the parameter w_{ij}**

$$w_{ij} = w_{ij} - \eta \frac{\partial E}{\partial w_{ij}} \qquad \frac{\partial E}{\partial w_{ij}} = a_i \delta_j$$

- **Update the biases** (take b_j as example)

$$b_j = b_j - \eta \frac{\partial E}{\partial b_j} \qquad \frac{\partial E}{\partial b_j} = \delta_j$$



$$\delta_k = g_k'(z_k) E'(a_k, tk) = g_k'(z_k)(a_k - tk)$$

$$\delta_j = g_j'(z_j) \sum_{\text{out}} \delta_k \, w_{jk}$$

★ η *is the learning rate (step size)*

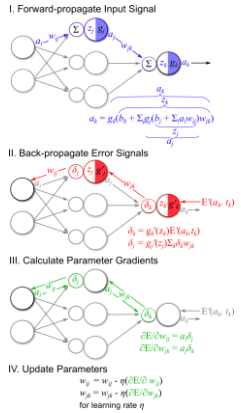
1 source: https://towardsdatascience.com/step-by-step-the-math-behind-neural-networks-490dc1f3cfd9

:: Neural Network

Summary of Backpropagation algorithm

Backpropagation algorithm

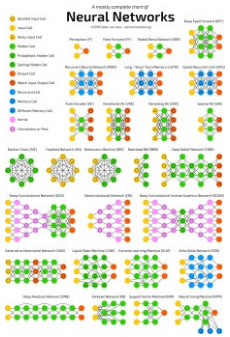
1. Initialize the network's weights randomly
2. **Implement forward propagation to get network output**
3. **Compute the error (loss function)**
4. **And then back propagates the error to the previous layers**
5. **These previous layers then adjust their weights and biases accordingly.**
6. Repeat this process (from step 2) to minimize the error until the loss converges



:: Neural Network

Overview of Neural Network Architectures [1]

- **Perceptron**—Neural Network having two input units and one output units with no hidden layers. These are also known as 'single layer perceptron.
- **Radial Basis Function Network**—These networks are similar to the feed forward neural network except radial basis function is used as activation function of these neurons.
- **Multilayer Perceptron**—These networks use more than one hidden layer of neurons, unlike single layer perceptron. These are also known as deep feedforward neural networks.
- **Recurrent Neural Network (RNN)**—Type of neural network in which hidden layer neurons has self-connections. Recurrent neural networks possess memory. At any instance, hidden layer neuron receives activation from the lower layer as well as it previous activation value.
- **Long /Short Term Memory Network (LSTM)**—Type of neural network in which memory cell is incorporated inside hidden layer neurons is called LSTM network.
- **Hopfield Network**—A fully interconnected network of neurons in which each neuron is connected to every other neuron. The network is trained with input pattern by setting a value of neurons to the desired pattern. Then its weights are computed. The weights are not changed. Once trained for one or more patterns, the network will converge to the learned patterns. It is different from other neural networks.
- **Boltzmann Machine Network**—These networks are similar to Hopfield network except some neurons are input, while other are hidden in nature. The weights are initialized randomly and learn through back propagation algorithm.
- **Convolutional Neural Network (CNN)**—A CNN consists of an input and an output layer, as well as multiple hidden layers. The hidden layers of a CNN typically consist of convolutional layers, pooling layers, fully connected layers and normalization layers[2]



Details can be found on
http://www.asimovinstitute.org/neural-network-zoo/

1 source: https://www.xenonstack.com/blog/overview-of-artificial-neural-networks-and-its-applications
2 source https://en.wikipedia.org/wiki/Convolutional_neural_network

:: Neural Network

Application of Neural Network

- **Classification**
  A neural network can be trained to classify given pattern or data set into predefined class. It uses feedforward networks.[1]
- **Prediction**
  A neural network can be trained to produce outputs that are expected from given input. Eg:—Stock market prediction.[2]
- **Clustering**
  The Neural network can be used to identify a special feature of the data and classify them into different categories without any prior knowledge of the data. [3]

Application examples

- Character recognition
- Image recognition
- Face recognition
- Stock prediction
- Voice recognition
- Fraud Detection
- Sales Forecast
- Cash Flow Forecasting
- Credit Rating
- …

Source: https://medium.com/@xenonstack/overview-of-artificial-neural-networks-and-its-applications-2525c1addff7

:: Neural Network

Advantage of Neural Network

- **Prediction accuracy is generally high**
  - record-breaking accuracy on a whole range of problems including image and sound recognition, text and time series analysis, etc.
  - Can significantly out-perform other models when the conditions are right (lots of high quality labeled data).
- **Very powerful algorithm which can to learn and model non-linear and complex relationships**
  - Can approximate non-linear function
  - Adapt to unknown situations. Flexible for real world applications. Great forcomplex/abstract problems like image, voice recognition
  - Can handle large number of features, even thousands ofinputs
  - Once trained, the output are given quickly
- **Robust**
  - High tolerance to noisy data
- **Very hot algorithm in recent years**
  - large amount of academic research
  - used extensively in industry
  - lots of libraries / implementations available

:: Neural Network

Disadvantage of Neural Network

- **Black Box model**
  Difficult to interpret what the model has learned.
- **Usually it's hard to train and take long training time**
  - The structure of Deep neural network is complicated
  - Require tuning a number of hyper-parameters such as the number of hidden neurons, layers, and learning rate. Many hyper-parameters have to be determined empirically
  - The training is computationally expensive
  - Very data Hungary. Requires a lot of data to perform well.
- **Prone to overfitting**
  - There are lots of methods for reducing overfitting. Such as applying regularization term, Early Stopping, Dropout, etc
- **Don't perform well on small data sets**
  - There are alternatives that are simpler, faster, easier to train, and provide better performance (SVM, Decision Trees, Bayesian, etc).

:: Neural Network

The common failure cases

There are a number of common ways for backpropagation to go wrong.[1] The most annoying challenges in deep learning optimization might be local minima, saddle points and vanishing gradients, etc. Let's have a look at them.

Backpropagation algorithm can get stuck during gradient descent

The gradients could explode or vanish

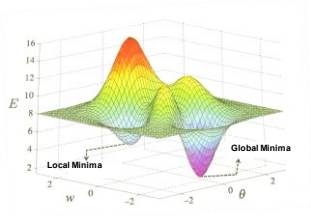
ReLU units can be fragile during training and can "die"

The neural network can easily overfit to training data

:: Neural Network

Backpropagation algorithm can get stuck during gradient descent

- **Gradient Descent may have trouble finding the global minimum**
  The error surface can have multiple local minimum. Gradient descent with backpropagation is not guaranteed to find the global minimum of the error function, but only a local minimum; also, it has trouble in crossing plateaus near saddle points in the error function landscape. This issue, is caused by the non-convexity of error functions in neural networks, was long thought to be a major drawback, but Yann LeCun et al. argue that in many practical problems, it is not.[1] "For large-size networks,... struggling to find the global minimum on the training set (as opposed to one of the many good local ones) is not useful in practice and may lead to overfitting [2] "



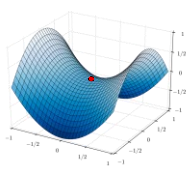
:: Neural Network

Backpropagation algorithm can get stuck during gradient descent – continued

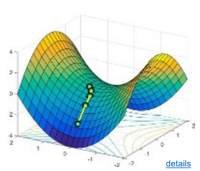
- **The prevalence of saddle points in high dimensional space is main difficulty for minimizing non-convex error functions**

  " Saddle points, not local minima, provide a fundamental impediment to rapid high dimensional non-convex optimization. … a deeper and more profound difficulty originates from the proliferation of saddle points, not local minima, especially in high dimensional problems of practical interest. Such saddle points are surrounded by high error plateaus that can dramatically slow down learning, and give the illusory impression of the existence of a local minimum." [2]

Saddle point



How to escape from Saddle point ?



details

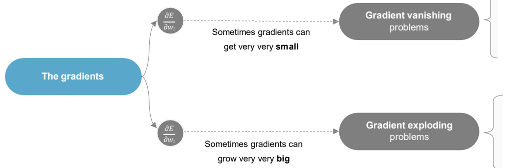
We'll go into the details of Saddle point later.

2 Source: https://arxiv.org/pdf/1412.0233.pdf
3 Source: https://arxiv.org/pdf/1406.2572.pdf

:: Neural Network

Another problem: the gradients could explode or vanish !!!

One of the problems of training neural network, especially very deep neural networks, is vanishing/exploding the gradients. In fact, for a long time this problem was a huge barrier to training deep neural networks. [1]



Vanishing gradients make it difficult to know which direction the parameters should move to improve the cost function.[2] In the worst case, this may completely stop the neural network from further training.[3]

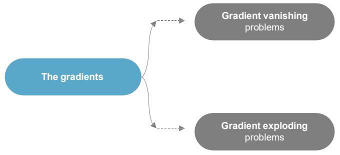
Exploding gradients can make learning unstable. At an extreme, the values of weights can become so large as to overflow and result in NaN values. Exploding gradients problem is mostly encountered in recurrent neural networks

1 Andrew ng, Deep Learning, https://www.youtube.com/watch?v=qhXZsFVxGKo
2 source: Ian Goodfellow, Deep learning
3 source: https://en.wikipedia.org/wiki/Vanishing_gradient_problem

:: Neural Network

More about Vanishing/Exploding Gradients Problems

The backpropagation algorithm works by going from the output layer to the input layer, propagating the error gradient on the way. Once the algorithm has computed the gradient of the cost function with regards to each parameter in the network, it uses these gradients to update each parameter with a Gradient Descent step[1]. In deep networks, computing these gradients can involve taking the product of many small/large terms.



The gradients (or derivatives) can sometimes get smaller and smaller
Unfortunately, gradients often get smaller and smaller as the algorithm progresses down to the lower layers. As a result, the Gradient Descent update leaves the lower layer connection weights virtually unchanged, and training never converges to a good solution. This is called the vanishing gradients problem [2]

The gradients (or derivatives) can sometimes grow bigger and bigger
When the error is back-propagated through a neural network, it may increase exponentially from layer to layer. In those cases, the gradient with respect to the parameters in lower layers may be exponentially greater than the gradient with respect to parameters in higher layers. This makes the network hard to train if it is sufficiently deep.[3]

1,2 Aurélien Géron ,Hands-On Machine Learning with Scikit-Learn and TensorFlow
3. The exploding gradient problem demystied - denition, prevalence, impact, origin, tradeos, and solutions

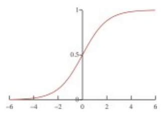
:: Neural Network

Why Gradients explode or vanish

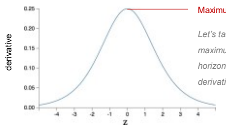
- **Vanishing gradients problems**

  One of the insights in the 2010 paper by Glorot and Bengio was that the vanishing/exploding gradients problems were in part due to a poor choice of activation function.[1] For example, sigmoid "squeezes" the real number line onto a "small" range of [0, 1]. As a result, there are large regions of the input space which are mapped to an extremely small range. In these regions of the input space, even a large change in the input will produce a small change in the output - **hence the gradient is small**.

Sigmoid function



Derivative of sigmoid function



Maximum value = 0.25

Let's take a look at the derivative of the sigmoid function. The maximum point of the sigmoid function is 1/4, and the function horizontally asymptotes at 0. In other words, the output of the derivative of the cost function is always between 0 and 1/4.

This becomes much worse when we stack multiple layers of such non-linearities on top of each other. [2] Because backpropagation computes gradients by the chain rule. This has the effect of **multiplying** ***n*** **of these small numbers** to compute gradients of the "front" layers in an *n*-layer network, meaning that the gradient (error signal) decreases exponentially with *n*. [3] So there is really nothing left for the front layers. When the gradients vanish toward 0 for the lower layers, these layers train very slowly, or not at all. The ReLU activation function can help prevent vanishing gradients.[4]

1 source: Aurélien Géron ,Hands-On Machine Learning with Scikit-Learn and TensorFlow
2 Source: https://www.quora.com/What-is-the-vanishing-gradient-problem
3source: https://en.wikipedia.org/wiki/Vanishing_gradient_problem#cite_note-17
4 source: https://developers.google.com/machine-learning/crash-course/training-neural-networks/best-practices

:: Neural Network

Why Gradients explode or vanish - *continued*

- **Exploding gradients problems**

  In deep networks, computing these gradients can involve taking the product of many small/large terms. The explosion occurs through exponential growth by **repeatedly multiplying gradients** through the network layers that have values larger than 1.0 [2] Therefore, the gradients can grow bigger and bigger, so many layers get insanely large weight updates and the algorithm diverges.[3]

  In other words, If the weights in a network are very large, then the gradients for the lower layers involve products of many large terms. In this case you can have exploding gradients: gradients that get too large to converge.

  Batch normalization can help prevent exploding gradients, as can lowering the learning rate.

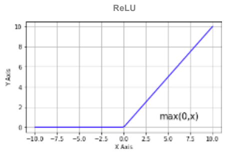
Video: Vanishing/Exploding Gradients by Andrew Ng (6mins)
https://www.youtube.com/watch?v=qhXZsFVxGKo

1 ,3 source: Aurélien Géron ,Hands-On Machine Learning with Scikit-Learn and TensorFlow
2 Source: https://machinelearningmastery.com/exploding-gradients-in-neural-networks/

:: Neural Network

Dead ReLU Units issue

.



$f(x) = \max(0, x)$

This means that when the input x < 0 the output is 0 and if x > 0 the output is x.

Once the weighted sum for a ReLU unit falls below 0, the ReLU unit can get stuck It outputs 0 activation, contributing nothing to the network's output, and gradients can no longer flow through it during backpropagation. With a source of gradients cut off, the input to the ReLU may not ever change enough to bring the weighted sum back above 0.[1]

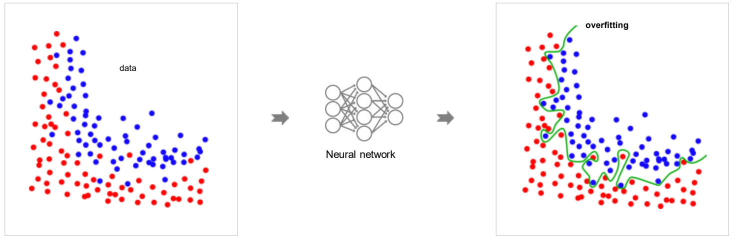
Lowering the learning rate can help keep ReLU units from dying.[2]

1&2 Source: https://developers.google.com/machine-learning/crash-course/training-neural-networks/best-practices

:: Neural Network

Neural networks is prone to overfitting the training set

Deep neural networks typically have tens of thousands of parameters, sometimes even millions. With so many parameters, the network has an incredible amount of freedom and can fit a huge variety of complex datasets. But this great flexibility also means that it is prone to overfitting the training set.



Source of image : https://en.wikipedia.org/wiki/Overfitting#/media/File:Overfitting.svg

:: Neural Network

Common approaches to reduce overfitting in neural network

There are a variety of techniques for reducing overfitting issue. For example, Get more Data, Augment data , Use "Bagging", Use less complicated architecture (e.g. less hidden layers, less nodes per layer), Noise injection, use Regularization, Use Dropout, etc. Typically, a combination of several these methods is used. [1]

Some of popular approaches

- **Early Stopping**
  Start with small weights and stop the learning before it overfits [2]
- **L1 & L2 Regularization**
  Penalize large weights using penalties or constraints on their squared values (L2 penalty) or absolute values (L1 penalty) [3]
- **Dropout Regularization**
  randomly dropping neurons from the network during training.

1 Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow
2, 3 Source: Geoffrey Hinton, https://www.youtube.com/watch?v=W0SP8FTmGW0

:: Neural Network

Common approaches to reduce overfitting in neural network

There are a variety of techniques for reducing overfitting issue. For example, Get more Data, Augment data , Use "Bagging", Use less complicated architecture (e.g. less hidden layers, less nodes per layer), Noise injection, use Regularization, Use Dropout, etc. Typically, a combination of several these methods is used. [1]

Some of popular approaches

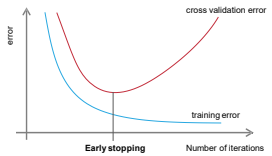
- **Early Stopping**
  Start with small weights and stop the learning before it overfits [2]
- **L1 & L2 Regularization**
  Penalize large weights using penalties or constraints on their squared values (L2 penalty) or absolute values (L1 penalty) [3]
- **Dropout Regularization**
  randomly dropping neurons from the network during training.

1 Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow
2, 3 Source: Geoffrey Hinton, More: www.youtube.com/watch?v=i6G2P8F7mGW0

:: Neural Network

Reduce overfitting by Early Stopping

To avoid overfitting the training set, a great solution is early stopping : just interrupt training when its performance on the validation set starts dropping.



- If we have lots of data and a big model, it's very expensive to keep re-training it with different sized penalties on the weights.
- It's much cheaper to **start with very small weights and let them grow until the performance on the validation set starts getting worse**.
  - But it ca be hard to decide when performance is getting worse.
- The capacity of the model is limited because the weights have not had time to grow big

One way to implement Eealy Stopping with TensorFlow is to evaluate the model on a validation set at regular intervals (e.g., every 50 steps), and save a "winner" snapshot if it outperforms previous "winner" snapshots. Count the number of steps since the last "winner" snapshot was saved, and interrupt training when this number reaches some limit (e.g., 2,000 steps). Then restore the last "winner" snapshot.

1 Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow

:: Neural Network

Common approaches to reduce overfitting in neural network

There are a variety of techniques for reducing overfitting issue. For example, Get more Data, Augment data , Use "Bagging", Use less complicated architecture (e.g. less hidden layers, less nodes per layer), Noise injection, use Regularization, Use Dropout, etc. Typically, a combination of several these methods is used. [1]

Some of popular approaches

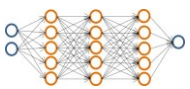
- **Early Stopping**
  Start with small weights and stop the learning before it overfits [2]
- **L1 & L2 Regularization**
  Penalize large weights using penalties or constraints on their squared values (L2 penalty) or absolute values (L1 penalty) [3]
- **Dropout Regularization**
  randomly dropping neurons from the network during training.

1 Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow
2, 3 Source: Geoffrey Hinton, More: www.youtube.com/watch?v=i6G2P8F7mGW0

:: Neural Network

Reduce overfitting by applying L2 Regularization (Weight Decay)

You can use ℓ_1 and ℓ_2 regularization to constrain a neural network's connection weights (but typically not its biases) [1]. L2 regularization is perhaps the most common form of regularization



simply add the L2 penalty to your cost function

L2 regularization is the most common type of regularization. It can be implemented by penalizing the **squared** magnitude of all parameters directly in the objective. That is, for every weight w in the network, we add the term $\frac{1}{2}\lambda w^2$ to the objective, where λ is the regularization strength. [2]

$$J(\Theta) = -\frac{1}{m}\left[\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(h_\Theta(x^{(i)})\right)_k + \left(1 - y_k^{(i)}\right) \log\left(1 - \left(h_\Theta(x^{(i)})\right)_k\right)\right] + \underbrace{\frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_l+1}\left(\Theta_{ji}^{(l)}\right)^2}_{\text{Regularization term}}$$

A regularization term is added to a loss function.
It is typically chosen to impose a penalty on the complexity of predicting function.

Video: Why Regularization Reduces Overfitting by Andrew Ng? (7 mins)
https://www.youtube.com/watch?v=NyG-7nRpsW8

1 Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow
2 Source: http://cs231n.github.io/neural-networks-2/#reg

:: Neural Network

Common approaches to reduce overfitting in neural network

There are a variety of techniques for reducing overfitting issue. For example, Get more Data, Augment data , Use "Bagging", Use less complicated architecture (e.g. less hidden layers, less nodes per layer), Noise injection, use Regularization, Use Dropout, etc. Typically, a combination of several these methods is used. [1]

Some of popular approaches

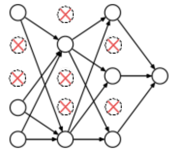
- **Early Stopping**
  Start with small weights and stop the learning before it overfits [2]
- **L1 & L2 Regularization**
  Penalize large weights using penalties or constraints on their squared values (L2 penalty) or absolute values (L1 penalty) [3]
- **Dropout Regularization**
  randomly dropping neurons from the network during training.

1 Source: Hands-On Machine Learning with Scikit-Learn & TensorFlow
2, 3 Source: Geoffrey Hinton, https://www.youtube.com/watch?v=W0SP8FTmGW0

:: Neural Network

Reduce overfitting by applying Dropout Regularization

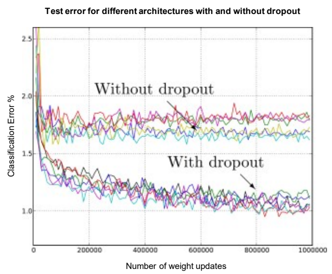
Dropout training simulates the notion of zero weights by eliminating some of the hidden units while training. In dropout training, each time an example is read, some hidden units are removed with probability , and the network is trained and propagates error as if those weights were not there. [1]



Each time we present a training example, we randomly omit each unit with probability [2] (e.g. 0.5)

Applying dropout to a neural network amounts to **sampling a "thinned" network** from it [3]

During testing there is no dropout applied, with the interpretation of evaluating an averaged prediction across the exponentially-sized ensemble of all sub-networks [4]



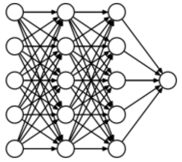
The networks have 2 to 4 hidden layers each with 1024 to 2048 units.

1 Source: Dan Roth, CS 446 Machine Learning Fall 2016
2 Source: Geoffrey Hinton ,https://www.youtube.com/watch?v=kAwF--GJ-ek&t=166s
3 Sourc & image : Source: Nitish Srivastava,et al., Dropout: A Simple Way to Prevent Neural Networks from Overfitting
4 Source: http://cs231n.github.io/neural-networks-2/

:: Neural Network

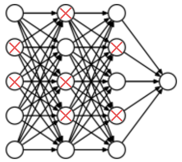
Understand how Dropout works

Standard backpropagation learning builds up brittle co-adaptations that work for the training data but do not generalize to unseen data. Random dropout breaks up these co-adaptations making the presence of any particular hidden unit unreliable.[1]



Standard Neural Net

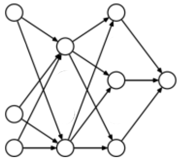
If a hidden unit know which other units are present, it can co-adapt to them on the training data. But complex co-adaptions are likely to go wrong on new test data.



Apply dropout

At each training stage, individual nodes are either dropped out of the net with probability $1-p$ or kept with probability p

The choice of which units to drop is random.[2]



After applying dropout

A reduced network which is smaller is left.

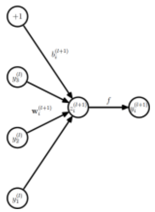
By dropping a unit out, we mean temporarily removing it from the network, along with all its incoming and outgoing connections.[3]

1, 2,3 Source: Nitish Srivastava,et al., Dropout: A Simple Way to Prevent Neural Networks from Overfitting
Source of image: Srivastava et al ,Dropout: A Simple Way to Prevent Neural Networks from Overtting

:: Neural Network

Understand how Dropout works (continued)

Consider a neural network with L hidden layers. Let $z^{(a)}$ denote the vector of inputs into layer l, $y^{(a)}$ denote the vector of outputs from layer l



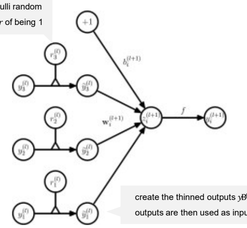
Standard network

The feed-forward operation of a standard neural network can be described as (for $l \in \{0, \dots, l-1\}$ any hidden unit i)

$$z_i^{(l+1)} = \mathbf{w}_i^{(l+1)} \mathbf{y}^l + b_i^{(l+1)},$$
$$y_i^{(l+1)} = f(z_i^{(l+1)}),$$

where f is any activation function

$r^{(T)}$ is a vector of independent Bernoulli random variables each of which has probability r of being 1



create the thinned outputs $y B^{(l)}$. The thinned outputs are then used as input to the next layer.

Dropout network

With dropout, the feed-forward operation becomes

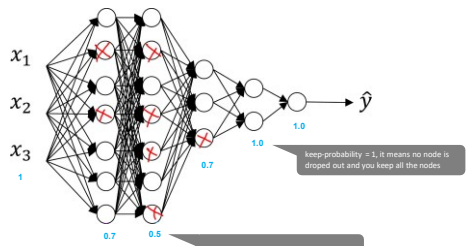
$$r_j^{(l)} \sim \text{Bernoulli}(p),$$
$$\tilde{\mathbf{y}}^{(l)} = \mathbf{r}^{(l)} * \mathbf{y}^{(l)},$$
$$z_i^{(l+1)} = \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^l + b_i^{(l+1)},$$
$$y_i^{(l+1)} = f(z_i^{(l+1)}).$$

Source: Nitish Srivastava,et al., Dropout: A Simple Way to Prevent Neural Networks from Overfitting

:: Neural Network

Different layers could have different probabilities for keeping units/nodes in network

Dropout has a tunable hyperparameter p (the probability of retaining a unit in the network). These could be different "keep_probability" p for different layers. If you're more worried about some layers overfitting than others, you can set a lower keep_prob for some layers than others.[1]



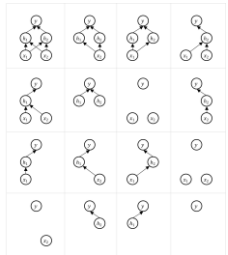
[1] Image Source: Andrew Ng, refer to www.youtube.com/watch?v=ARq74Qavqv0

:: Neural Network

Dropout can be viewed as a form of model averaging

The central idea of dropout is to take a large model that overfits easily and repeatedly sample and train smaller sub-models from it. It prevents overfitting and provides a way of approximately combining exponentially many different neural network architectures efficiently.[1]


Base network


Ensemble of subnetworks

There are 16 possible combinations, of which only 9 are viable. Non-viable are (6, 7, 12, 13, 14, 15, 16).

- **Model combination nearly always improves the performance of machine learning methods**
  Another way to view the dropout procedure is as a very efficient way of performing model averaging with neural networks. The standard way to do this is to train many separate networks and then to apply each of these networks to the test data, but this is computationally expensive during both training and testing.

  Random dropout makes it possible to train a huge number of different networks in a reasonable time. There is almost certainly a different network for each presentation of each training case but all of these networks share the same weights for the hidden units that are present.[2]

- **The sharing of the weights means that every model is very strongly regularized**
  Dropout can be seen as an extreme form of bagging in which each model is trained on a single case and each parameter of the model is very strongly regularized by sharing it with the corresponding parameter in all the other models. [3] This is a much better regularizer thanL2 or L1 penalties that pull the weights towards zero.

- **Dropout also has drawback**
  One of the drawbacks of dropout is that it increases training time. A dropout network typically takes 2-3 times longer to train than a standard neural network of the same architecture [4]

1,2,3,4 Source: Nitish Srivastava et al., Dropout: A Simple Way to Prevent Neural Networks from Overfitting, http://jmlr.csail.mit.edu/papers/volume15/srivastava14a/srivastava14a.pdf?utm_content=buffer79ad1&utm_medium=social&utm_source=twitter.com&utm_campaign=buffer